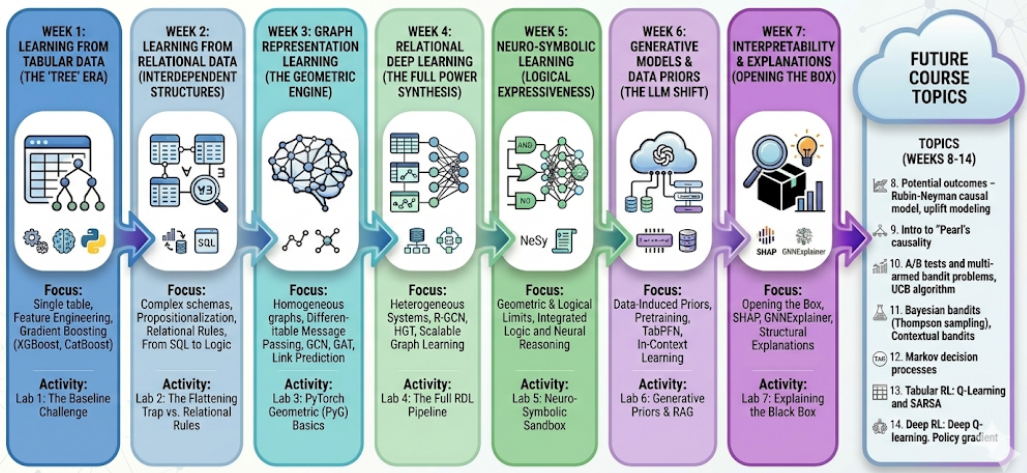


LEARNING FROM STRUCTURED DATA: A 7-WEEK JOURNEY

Narrative Arc: Increasing Structure → Increasing Inductive Bias → Increasing Reasoning Power



Course Map of Week 4: Relational Deep Learning

- 1 The Database Challenge
- 2 Pure Topology (Recommender Systems)
- 3 Semantic Facts (Knowledge Graphs)
- 4 Heterogeneous Message Passing
- 5 The Relational Deep Learning Blueprint
- 6 Pragmatic MLOps

The Impedance Mismatch: Deep Learning vs. Databases

The “Dark Matter” of AI

- Relational DBs power $\sim 90\%$ of industrial AI.
- Modern DL heavily favors perceptual data.

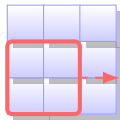
The Architectural Prior Gap

- **Images (CNNs):** Spatial grids (sliding windows).
- **Text (LLMs/RNNs):** 1D Sequences (causality).
- **Databases:** Heterogeneous dynamic sets.
No spatial locality!

The Challenge

- Standard DL architectures fail natively on SQL schemas due to misaligned topological priors.

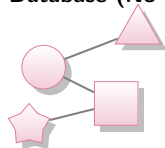
1. Image (CNN Prior)



2. Text (RNN Prior)



3. Database (No Prior)



Where is the spatial locality?

The “Flattening” Trap & Manual Feature Engineering

The Classical Hack: Propositionalization

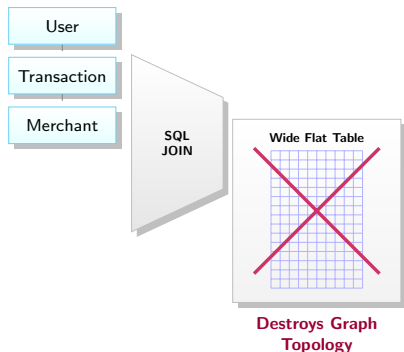
- Historically, we flattened a multi-table DB into a single matrix $X \in \mathbb{R}^{N \times F}$ to bypass the mismatch.

The “Topological Blindness” Limit

- Loss of Info:** Aggregations wash out topological signals.
- Redundancy:** SQL JOINS cause memory duplication.

The Human Bottleneck

- Requires (manual) Feature Engineering.
- High risk of temporal data leakage.



Engagement Question:

If flattening destroys structure, and standard DL fails natively, how do we solve this?

The Journey So Far: Towards the Grand Unification

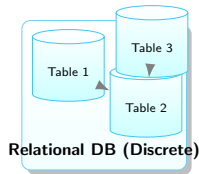
Week 1: The Tabular Baseline

- **Assumption:** i.i.d. rows.
- **Tool:** XGBoost on flat matrices.



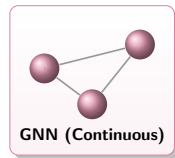
Week 2: The Relational Schema

- **Assumption:** Interacting entities.
- **Tool:** First-Order Logic & Propositionalization.



Week 3: The Geometric Engine

- **Assumption:** Continuous topology.
- **Tool:** GNNs on homogeneous graphs $G(V, E)$.



Bridging to Reality: The Simplest Relational Setup

Leaving the Homogeneous Sandbox

- Week 3 modeled single entity types (e.g., just 'People').
- Real databases have *multiple* tables.

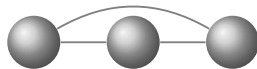
Step 1: The Bipartite Graph

- The simplest multi-table extension.
- Exactly **two** node types (tables) and **one** edge type (foreign key constraint).

Real-World Example

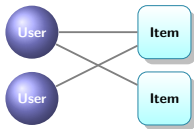
- Users $\leftrightarrow$ Movies
- This defines **Recommender Systems**.

Week 3: Homogeneous $G(V, E)$



Split Node Types

Week 4: Bipartite Heterogeneous



The Collaborative Filtering Paradigm

What is Collaborative Filtering (CF)?

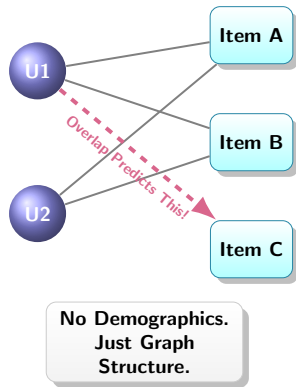
- Core philosophy: *"Users who agreed in the past will agree in the future."*

The Challenge of Pure Topology

- Standard Tabular ML (Week 1) relies on rich features (Age, Salary, Genre).
- CF assumes **zero features**. Nodes are just arbitrary IDs!

The Solution Space

- We must predict user preferences based entirely on *network overlap*.



The GNN Baseline (NGCF) & The Ablation Discovery

Neural Graph Collaborative Filtering (NGCF)

- Applied standard GNNs to the User-Item bipartite graph.

$$e_u^{(k+1)} = \cancel{\sigma} \left(\cancel{W} \cdot \text{AGG}(e_i^{(k)}) \right)$$

Ablation: Removes Nonlinearities & Weights!

The Ablation Discovery

- Feature transformations and non-linearities contribute *nothing* here.

The Inductive Bias Lesson

- When nodes only have arbitrary IDs, dense MLPs just add noise.

LightGCN: Simplifying the Convolution

The LightGCN Philosophy

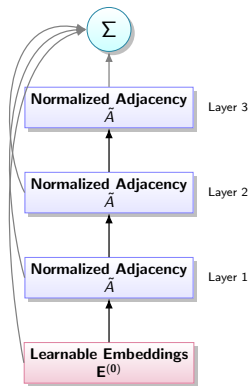
- Aggressively removes all learnable parameters from GNN layers.

Pure Neighborhood Aggregation

- Uses *only* linear neighborhood aggregation.
- Learnable parameters exist *only* in the initial shallow embeddings.

The Result

- Faster inference, easier to train, 16.0% relative improvement.



The Mathematics of LightGCN

Simplified Update Rule

- $e_u^{(k+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u| |\mathcal{N}_i|}} e_i^{(k)}$
- **Notice:** No W , no σ , no self-loop.

Matrix Formulation

- $E^{(k+1)} = \tilde{A} E^{(k)}$
- $\tilde{A}$ is normalized bipartite adjacency.

Final Readout

- $e_u = \frac{1}{K+1} \sum_{k=0}^K e_u^{(k)}$
- Averages all layers to mitigate over-smoothing.

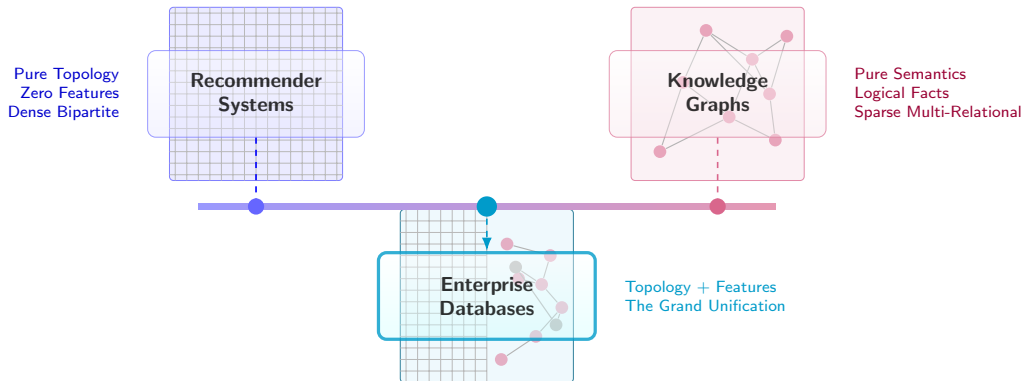
The diagram illustrates the matrix formulation of the update rule. It shows a vertical light blue rectangle on the left labeled $E^{(k+1)}$, followed by an equals sign, a light blue square in the center labeled $\tilde{A}$ with "(Sparse)" below it, followed by a multiplication symbol $\times$, and a vertical light blue rectangle on the right labeled $E^{(k)}$.

The diagram shows the final readout formula inside a rounded pink rectangle: $e_u = e_u^{(0)} + e_u^{(1)} + e_u^{(2)} + \dots$

The Relational Spectrum

Two Extremes of Relational Data

- We have just seen pure **Topology**. Next, we explore pure **Semantics**.



Knowledge Graphs: The Semantic Web

The Knowledge Graph (KG)

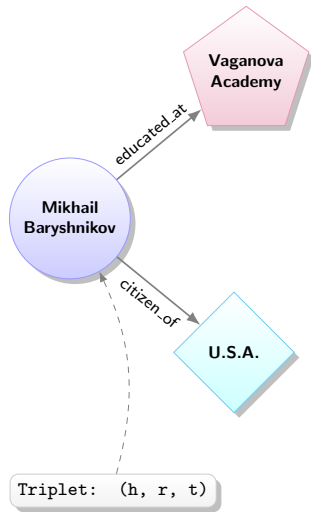
- A heterogeneous graph focusing on symbolic facts.
- **Vertices** = Entities (e.g., *Baryshnikov*).
- **Edges** = Relations (e.g., *educated_at*).

The Triplet Formalism

- Every fact is a directed tuple: $e = (h, r, t)$.
- Head (h), Relation (r), Tail (t).

Semantic Impedance Mismatch

- KGs are discrete and symbolic.
- *Goal*: Map discrete logical entities into continuous vector spaces.



The Task: Link Prediction (KG Completion)

The Incompleteness Problem

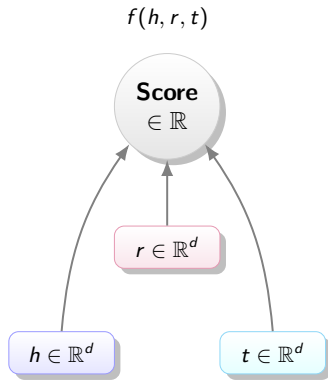
- KGs (e.g., Wikidata) are fundamentally incomplete.
- *Task: Link Prediction* (missing triples).

Encoder-Decoder Framework

- **Encoder:** Maps $v_i \in V$ to vector $e_i \in \mathbb{R}^d$.
- **Decoder:** Assigns a plausibility score to (h, r, t) .

Mathematical Objective

- $score = f(h, r, t)$
- Maximize scores for true facts; minimize for false facts.



TransE: The Translational Paradigm

Translating Embeddings (TransE)

- Baseline assumption: Relations are geometric translations.

The Geometric Equation

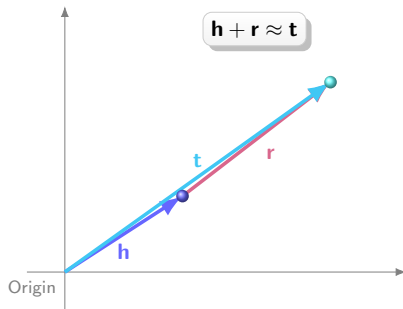
- If fact (h, r, t) holds true: $h + r \approx t$.
- Tail t is the nearest neighbor of $h + r$.

Hierarchical Inductive Bias

- Translations perfectly model trees and hierarchies (e.g. parent $\rightarrow$ child).

Scoring Function

- $f_r(h, t) = -||h + r - t||_{1/2}$



Training TransE: Margin-Based Ranking Loss

The Optimization Goal

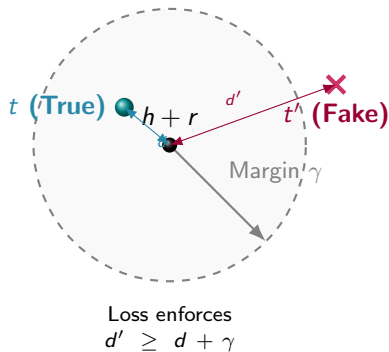
- Push valid triplets (h, r, t) closer.
- Push corrupted triplets (h', r, t') further apart.

The Loss Function

- $\mathcal{L} = \sum_S \sum_{S'} \max(0, \gamma + d(h + r, t) - d(h' + r, t'))$
- $\gamma > 0$ is the margin hyperparameter.

Negative Sampling

- Replace h or t with random entities to build S' .
- **Constraint:** Entity norms are bounded ($\|h\|_2 = 1$) to prevent artificial loss minimization.



Engagement Question: The Flaw of TransE

Engagement Question:

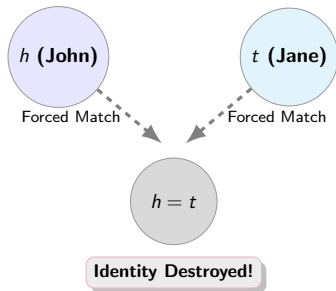
*If a relation is symmetric—both (h, r, t) AND (t, r, h) are true (e.g., *is_married_to*)—what happens to the vector r in TransE?*

A: The Mathematical Proof

- $h + r = t$ and $t + r = h$
- Substitute:
 $(h+r)+r = h \implies 2r = 0 \implies \mathbf{r = 0}$

The Consequence

- If $r = 0$, then $h = t$. TransE forces symmetric entities to have the *exact same embedding*.



If $(John, child, Mary)$ and $(John, child, Bob)$, Mary and Bob are forced identical!

DistMult: Semantic Matching via Bilinear Products

From Addition to Multiplication

- Replaces geometric translation with *bilinear mapping functions*.

The DistMult Scoring Function

- $f(h, r, t) = h^T M_r t$
- $M_r \in \mathbb{R}^{d \times d}$ is a **diagonal matrix** specific to relation r .

Solving the Symmetry Problem

- Diagonal math is commutative:
 $h^T M_r t = t^T M_r h$.
- Models symmetric relations perfectly without collapsing space.

$$\begin{matrix} h^T & M_r \text{ (Diag)} & t \\ \begin{bmatrix} h_1 & h_2 & h_3 \end{bmatrix} & \begin{bmatrix} r_1 & 0 & 0 \\ 0 & r_2 & 0 \\ 0 & 0 & r_3 \end{bmatrix} & \begin{bmatrix} t_1 \\ t_2 \\ t_3 \end{bmatrix} \end{matrix}$$
$$= \text{Scalar Score}$$
$$\sum_i h_i \cdot r_i \cdot t_i$$

Bridging to Logic: Mining Rules from Vectors

Why Bilinear Matters

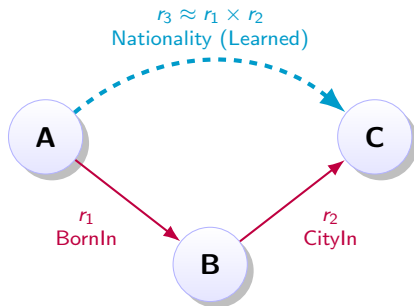
- Matrix operations organically encode logical rules.

Relation Composition

- Logical paths equal multiplying relation matrices.
- $M_{r_1} \times M_{r_2} \approx M_{r_3}$

Transitivity Example

- If $M_{BornIn} \times M_{CityIn} \approx M_{Nationality} \dots$
- The model learns the Horn Clause:
 $BornIn(a,b) \wedge CityIn(b,c) \implies Nationality(a,c).$



Beyond the Homogeneous Sandbox

Synthesizing Topology and Semantics

- Real databases have both rich features and diverse topological structures.

The Real World: Multi-Relational Data

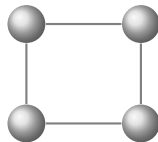
- Edges represent *different* types of interactions (e.g., 'Likes' vs. 'Follows').
- Extended definition: $G = (V, E, R)$.

The Mathematical Shift

- Edges become triplets: $e = (u, r, v)$.
- Adjacency becomes a Tensor:
 $\mathcal{A} \in \mathbb{R}^{|V| \times |R| \times |V|}$.

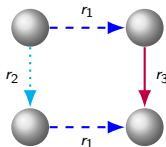
Homogeneous

$$A \in \mathbb{R}^{|V| \times |V|}$$



Multi-Relational

$$\text{Tensor } \mathcal{A} \in \mathbb{R}^{|V| \times |R| \times |V|}$$



Formalizing the Heterogeneous Graph

Formalizing Heterogeneous Graphs

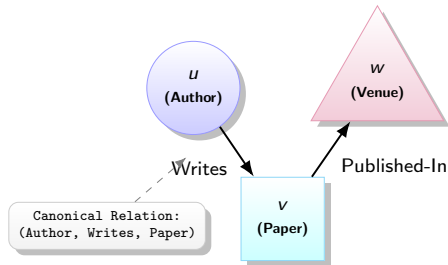
Heterogeneous Graph Definition: $G = (V, E, \tau, \phi)$

Node and Edge Typings

- **Nodes (τ):** Maps to node type (e.g., *Author*).
- **Edges (ϕ):** Maps to edge type (e.g., *Writes*).

Canonical Edge Types (Relations)

- Binds types: (Source, Edge, Destination)
- **Tuple:** $r(u, v) = (\tau(u), \phi(u, v), \tau(v))$
- **Example:** (Author, Writes, Paper)



The Metagraph (Graph Schema)

What is a Metagraph?

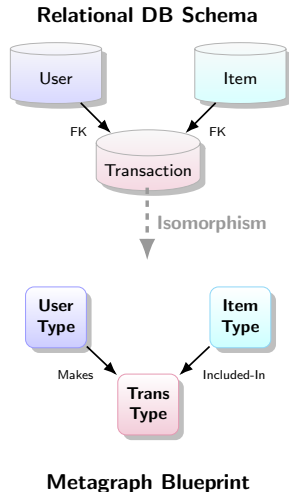
- “Blueprint” of a heterogeneous graph.
- **Nodes** = Node Types (T).
- **Edges** = Canonical Relations (R).

Schema to Graph Isomorphism

- A database schema *is* exactly a metagraph.
- It explicitly dictates structural constraints.

Type Constraints

- Edges must strictly obey the blueprint.
- e.g., ‘Purchases’ cannot connect ‘User’ to ‘City’.



The Core Problem: The Heterogeneous Mismatch & Requirement

Homogeneous GCN Limit

- Applies one shared weight W to *all* neighbors.

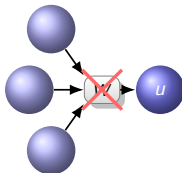
The Semantic Mismatch

- **Question:** Should a 'Purchased' edge be mathematically identical to a 'Lives-In' edge?
- **Answer:** No. They have different feature dimensions & predictive signals.

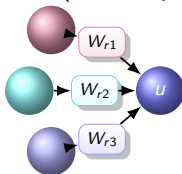
Architectural Requirement

- Abandon the shared W .
- Learn distinct, relation-specific routing channels (W_r).

Homogeneous GCN
(Shared W)



Relational GNN
(Relation-Specific W_r)



Relational Graph Convolutional Networks (R-GCN)

The R-GCN Layer Update (Schlichtkrull et al., 2018)

$$h_i^{(l+1)} = \sigma \left(\sum_{r \in R} \sum_{j \in \mathcal{N}_r(i)} \frac{1}{c_{i,r}} W_r^{(l)} h_j^{(l)} + W_0^{(l)} h_i^{(l)} \right)$$

Relation-Specific Aggregation

- **Outer sum** ($\sum_{r \in R}$): Iterates over relation types.
- **Inner sum** ($\sum_{j \in \mathcal{N}_r(i)}$): Aggregates neighbors connected via relation r .

The Transformations

- $W_r^{(l)}$: The distinct weight matrix for relation r at layer l .
- Learns independent semantic channels.

Deconstructing the R-GCN Engine

1. Degree Normalization ($c_{i,r}$)

- Prevents high-degree hubs from exploding gradients.

2. The Self-Connection ($W_0^{(l)}$)

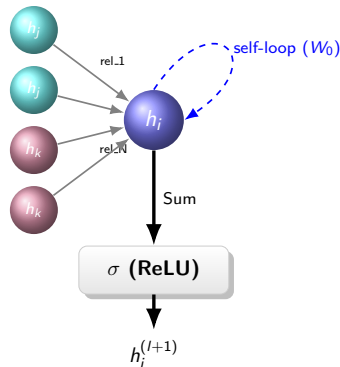
- Uses dedicated matrix W_0 to carry own state forward.

3. Non-Linearity (σ)

- Standard activation (e.g., ReLU) on combined message.

Vectorized Execution

- Computed via efficient sparse matrix ops.



The Scalability Crisis: Parameter Explosion

The Flaw of W_r

- A distinct matrix W_r for every relation triggers parameter explosion.

Engagement Question:

If our database has $|R| = 1,000$ and hidden dim $d = 512$, how many weights in one layer?

The Consequence

- **Memory Exhaustion:** Prohibitive scale.
- **Overfitting:** Rare relations lack training edges to update huge matrices.

$$1,000 \times (512 \times 512)$$

$$\approx$$

**262 Million
Parameters!**

(Per Layer)

Regularization 1: Basis Decomposition

Weight Sharing via Basis Matrices

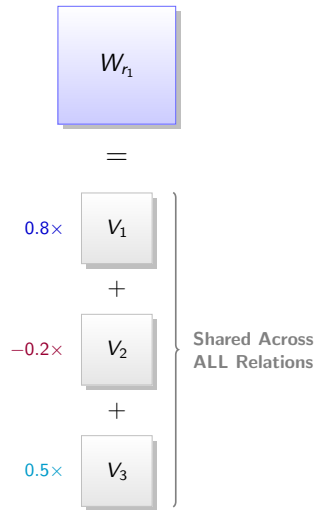
$$W_r^{(l)} = \sum_{b=1}^B a_{rb}^{(l)} V_b^{(l)}$$

The Mechanism

- V_b : Small set of *shared* basis matrices.
- a_{rb} : Relation-specific scalar coefficients.

Efficiency & Pedagogy

- Relations only learn B scalars instead of full matrices.
- Frequent relations update shared V_b , indirectly training rare relations.



Regularization 2: Block-Diagonal Decomposition

Enforcing Sparsity via Blocks

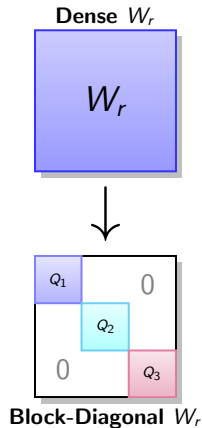
- $W_r^{(l)} = \bigoplus_{b=1}^B Q_{br}^{(l)}$
- Constrains W_r to be block-diagonal.

The Mechanism

- Constructed from low-dimensional matrices Q_{br} .

The Inductive Bias

- Exponentially reduces parameter count.
- Assumes features group into independent sets that don't interact *across* groups.



The Attention Upgrade: Heterogeneous Graph Transformers

Beyond Static Weights

- HGT (Hu et al., 2020) upgrades R-GCN to dynamic attention.

Meta-Relation Attention

- Weighs neighbor importance via Scaled Dot-Product Attention.

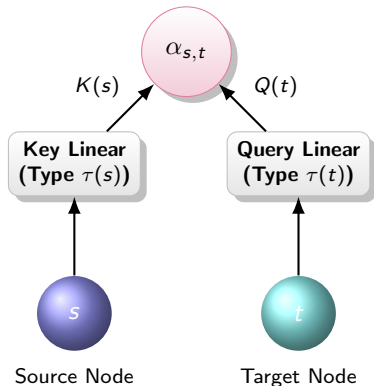
Type-Dependent Transformations

- Queries/Keys are based on **node type** $\tau(v)$.
- $K(s) = \text{Linear}_{\tau(s)}(H[s])$

The Result

- Maintains dedicated representations without parameter explosion.

Dynamic Attention Score



The Grand Unification: Relational Deep Learning (RDL)

The Paradigm Shift: Relational Deep Learning (RDL)

- End-to-end representation learning directly on relational DBs.
- *No manual feature engineering. No flattening.*

Step 1: Relational Graph

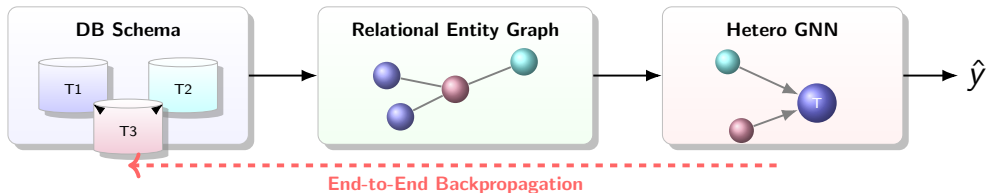
- Model entire DB schema as heterogeneous graph $G(V, E, R)$.

Step 2: Tabular Encoders

- Use MLPs as *feature extractors* for raw SQL rows.

Step 3: Message Passing

- Pass differentiable vectors across primary-foreign key edges.



The Relational Entity Graph (REG) Representation

Mapping Structure

- Map DB to heterogeneous graph.

Structural Isomorphism

- Table** $\rightarrow$ Node Type ($\tau \in T$)
- Row** $\rightarrow$ Node ($v \in V$)

Relational Isomorphism

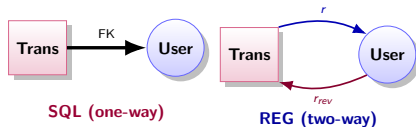
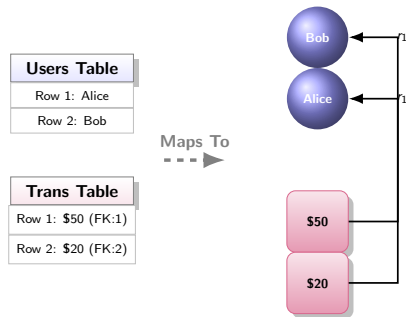
- Foreign Key** $\rightarrow$ Edge Type ($r \in R$)

Feature Isomorphism

- Columns** $\rightarrow$ Raw Node Features (X_v)

Practical Note

- For every edge type r , add a reverse r_{rev} .



The Second Impedance Mismatch: Features

GNN Input Requirement

- Every node v must be initialized with a dense vector: $h_v^{(0)} \in \mathbb{R}^d$.

The Reality of Rows

- Rows are highly multimodal:
- ["NY", 45.5, NaN, "2023-10-01"]

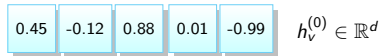
The Challenge

- How do we map raw SQL columns into a differentiable vector $h_v^{(0)}$ *without* manual feature engineering?

Raw SQL Row (Messy)



Required GNN Input



Engagement Question: The Return of Neural Nets

Engagement Question:

In Week 1, we proved tree-based models (XGBoost) crush Neural Networks on tabular rows. So why do we use Neural Networks to encode our tabular rows here?

A: The Gradient Constraint

- We are no longer building an isolated classifier.
- The row encoder acts as the **Input Layer** to a Graph Neural Network.

Differentiability is Mandatory

- Decision trees use hard, non-differentiable splits.
- We need gradients to flow backward from the GNN *through* the row encoder to optimize embeddings end-to-end.

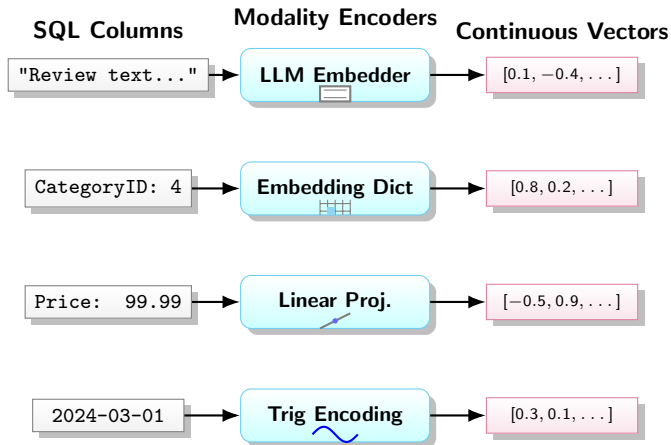
Deep Tabular Encoders: The First Layer

Deep Table Encoders

- Treats the SQL row as a multi-modal input.

Column-Level Modality Encoding

- **Text:** Pre-trained LLM embeddings.
- **Categorical:** Embedding Dictionaries.
- **Numerical:** Linear projections.



Row-Level Fusion ($h_v^{(0)}$)

From Columns to Nodes

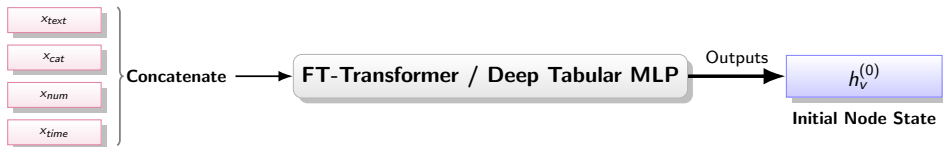
- Once columns are vectorized, we fuse them to represent the row.

The Fusion Mechanism

- Concatenate individual column embeddings.
- Pass through a deep tabular model (e.g., **FT-Transformer**).

The Output: $h_v^{(0)}$

- $h_v^{(0)}$: The initial geometric state of node v , ready for message passing.



The End-to-End RDL Architecture

Stage 1: Deep Table Encoders

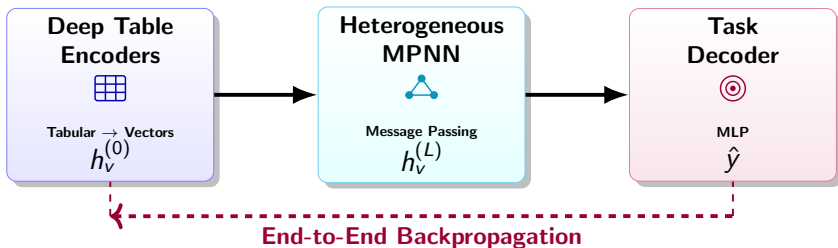
- Raw Rows $\rightarrow$ Initial Node Embeddings $h_v^{(0)}$

Stage 2: Relational Message Passing

- Embeddings $\rightarrow$ Contextualized Embeddings $h_v^{(L)}$

Stage 3: Task-Specific Head (Decoder)

- Contextualized Embeddings $\rightarrow$ Final Prediction $\hat{y}$



Standardizing the Tasks: RelBench

The Need for Standardization

- Relational DBs lacked a unified ML benchmark.
- **RelBench (2024)** standardizes RDL tasks.

Node-Level Tasks (Entity)

- Predict property of a row at a specific time.
- *Examples:* User Churn, LTV.

Link-Level Tasks (Edge)

- Predict the emergence of a future edge.
- *Examples:* User-Item Purchase.

The RDL Advantage

- Outperforms traditional XGBoost methods with **zero manual feature engineering**.

Classification Focus



Node Task:
Will this user churn?



Link Task:
Will edge form?

The Production Reality Check

The Theory is Complete

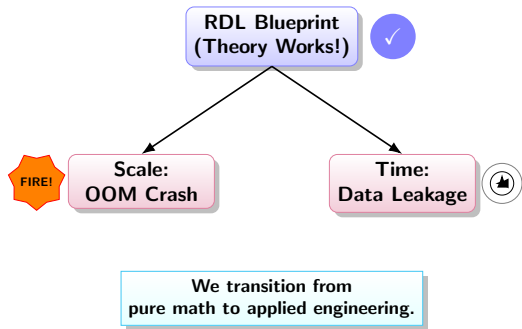
- We can map SQL to REG and route messages end-to-end.

Bottleneck 1: Scale

- Full-batch passing on 100M+ nodes causes **OOM** crashes.

Bottleneck 2: Time

- Databases are dynamic event logs.
- *Risk*: Data Leakage (predicting the future with future data).



Scalability: The Neighborhood Explosion

The Receptive Field Problem

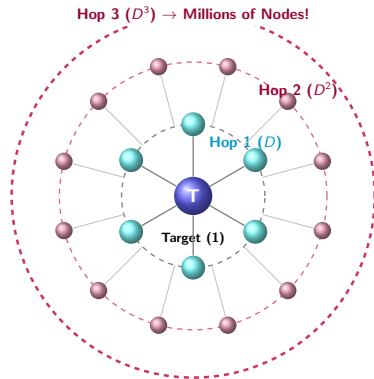
- A K -layer GNN needs K -hop neighbors for *one* target node.

Exponential Growth

- For average degree D :
- Layer 1: $D \rightarrow$ Layer K : D^K

The Consequence

- Predicting a single node on a dense hub requires loading millions of neighbors!



Mini-Batching via Neighbor Sampling

The Solution: Subgraph Sampling

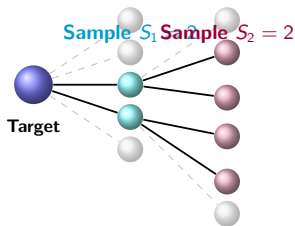
- Construct localized mini-batches instead of using full adjacency.

Top-Down Neighbor Sampling

- Randomly sample a **fixed max** of neighbors per layer (e.g., $S_1 = 15$).

The Computation Graph

- Transforms the tree into a strictly bounded, sparse Graph.



Strictly Bounded Memory!

Engagement Question: The Temporal Leakage Trap

Engagement Question:

*Predicting if User A churns on **June 1st**. GNN routes info from a 'Transaction' dated **June 5th**. What happened?*

A: Catastrophic Data Leakage ("Time Travel")

- Used future data to predict past event.
- Training accuracy: 99% → Production accuracy: Garbage.

The Core Concept: Seed Time (t_{seed})

- Every prediction strictly bound to a timestamp.
- Must sever edges violating causality.

Time-Consistent Computation Graphs

Temporal Subgraph Sampling

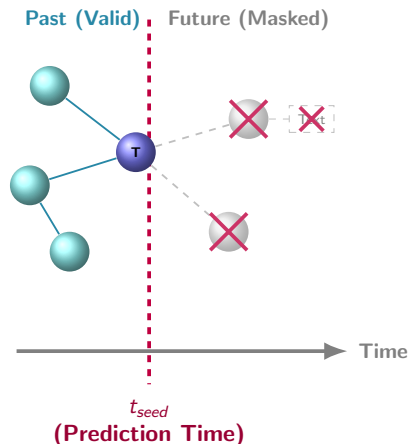
- Inject temporal logic directly into the NeighborLoader.

The Causality Constraint

- Only sample edge e if its timestamp $t_e \leq t_{seed}$.

End-to-End Temporality

- The graph structure changes depending on t_{seed} !
- Constraints **also apply to Tabular Encoders** (e.g., text generated *after* t_{seed} is masked).



Full Circle: RDL in the MLOps Pipeline

The Culmination

- Synthesizing Tabular, Relational, and Geometric data.

Deployment A: Pure End-to-End

- Decoder ($\hat{y}$) used directly. Zero feature engineering.

Deployment B: “Augmented Tabular”

- Feed Contextualized Embeddings $h_v^{(L)}$ as new columns **back into XGBoost!**

Why Option B?

- Combines topological reasoning of GNNs with statistical robustness of Trees.

